

Name : Nishagiri Kamble

Roll No: CO2032

Diabetes Prediction using Machine Learning with Python

Problem Stetement

he objective of this project is to build a Machine Learning model that predicts whether a person is diabetic or non-diabetic based on medical parameters such as pregnancies, glucose level, blood pressure, skin thickness, insulin level, BMI, diabetes pedigree function, and age. The model helps in early detection of diabetes, enabling timely medical intervention and better healthcare decisions

Libraries Imports

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Dataset Loading

```
In [3]: df=pd.read_csv("diabetes_prediction_dataset.csv")
```

```
In [4]: df
```

Out[4]:

	gender	age	hypertension	heart_disease	smoking_history	bmi	HbA1c_level	bl
0	Female	80.0	0	1	never	25.19	6.6	
1	Female	54.0	0	0	No Info	27.32	6.6	
2	Male	28.0	0	0	never	27.32	5.7	
3	Female	36.0	0	0	current	23.45	5.0	
4	Male	76.0	1	1	current	20.14	4.8	
...	...	...	...	...	...	...	...	...
99995	Female	80.0	0	0	No Info	27.32	6.2	
99996	Female	2.0	0	0	No Info	17.37	6.5	
99997	Male	66.0	0	0	former	27.83	5.7	
99998	Female	24.0	0	0	never	35.42	4.0	
99999	Female	57.0	0	0	current	22.43	6.6	

100000 rows × 9 columns



Dataset Information

In [5]:

```
df.head()
```

Out[5]:

	gender	age	hypertension	heart_disease	smoking_history	bmi	HbA1c_level	blood_g
0	Female	80.0	0	1	never	25.19	6.6	
1	Female	54.0	0	0	No Info	27.32	6.6	
2	Male	28.0	0	0	never	27.32	5.7	
3	Female	36.0	0	0	current	23.45	5.0	
4	Male	76.0	1	1	current	20.14	4.8	



In [6]:

```
df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   gender                 100000 non-null object
1   age                    100000 non-null float64
2   hypertension           100000 non-null int64
3   heart_disease          100000 non-null int64
4   smoking_history        100000 non-null object
5   bmi                    100000 non-null float64
6   HbA1c_level            100000 non-null float64
7   blood_glucose_level    100000 non-null int64
8   diabetes               100000 non-null int64
dtypes: float64(3), int64(4), object(2)
memory usage: 6.9+ MB

```

In [7]: `df.shape`

Out[7]: (100000, 9)

In [8]: `df.tail`

```

Out[8]: <bound method NDFrame.tail of
king_history  bmi \
0   Female  80.0      0      1      never  25.19
1   Female  54.0      0      0      No Info  27.32
2   Male    28.0      0      0      never  27.32
3   Female  36.0      0      0      current  23.45
4   Male    76.0      1      1      current  20.14
...   ...   ...      ...      ...      ...   ...
99995 Female  80.0      0      0      No Info  27.32
99996 Female   2.0      0      0      No Info  17.37
99997 Male    66.0      0      0      former  27.83
99998 Female  24.0      0      0      never  35.42
99999 Female  57.0      0      0      current  22.43

      HbA1c_level  blood_glucose_level  diabetes
0              6.6                140         0
1              6.6                 80         0
2              5.7                158         0
3              5.0                155         0
4              4.8                155         0
...   ...   ...      ...      ...
99995          6.2                 90         0
99996          6.5                100         0
99997          5.7                155         0
99998          4.0                100         0
99999          6.6                 90         0

[100000 rows x 9 columns]>

```

In [9]: `df.describe()`

Out[9]:

	age	hypertension	heart_disease	bmi	HbA1c_level	blood_g
count	100000.000000	100000.000000	100000.000000	100000.000000	100000.000000	100000.000000
mean	41.885856	0.07485	0.039420	27.320767	5.527507	100.000000
std	22.516840	0.26315	0.194593	6.636783	1.070672	100.000000
min	0.080000	0.00000	0.000000	10.010000	3.500000	100.000000
25%	24.000000	0.00000	0.000000	23.630000	4.800000	100.000000
50%	43.000000	0.00000	0.000000	27.320000	5.800000	100.000000
75%	60.000000	0.00000	0.000000	29.580000	6.200000	100.000000
max	80.000000	1.00000	1.000000	95.690000	9.000000	100.000000

In [12]: `df.dtypes`

```
Out[12]: gender          object
age              float64
hypertension     int64
heart_disease    int64
smoking_history  object
bmi              float64
HbA1c_level      float64
blood_glucose_level int64
diabetes         int64
dtype: object
```

In [13]: `df.columns`

```
Out[13]: Index(['gender', 'age', 'hypertension', 'heart_disease', 'smoking_history',
                'bmi', 'HbA1c_level', 'blood_glucose_level', 'diabetes'],
                dtype='object')
```

Data Cleaning

In [10]: `df.isnull().sum()`

```
Out[10]: gender          0
age              0
hypertension     0
heart_disease    0
smoking_history  0
bmi              0
HbA1c_level      0
blood_glucose_level 0
diabetes         0
dtype: int64
```

In [11]: `df.duplicated().sum()`

```
Out[11]: 3854
```

```
In [14]: df.drop_duplicates(inplace=True)
```

```
In [15]: df.duplicated().sum()
```

```
Out[15]: 0
```

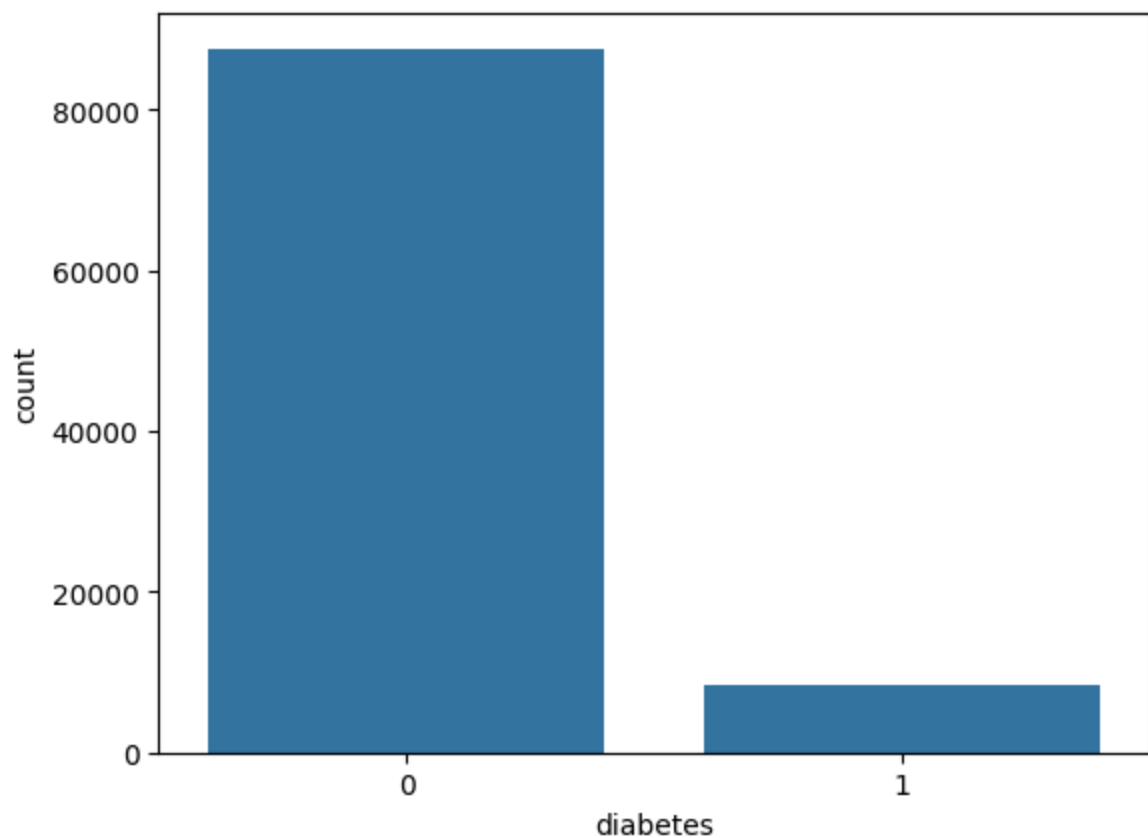
```
In [18]: df.columns
```

```
Out[18]: Index(['gender', 'age', 'hypertension', 'heart_disease', 'smoking_history',  
              'bmi', 'HbA1c_level', 'blood_glucose_level', 'diabetes'],  
              dtype='object')
```

```
In [19]: df["diabetes"].value_counts()
```

```
Out[19]: diabetes  
0      87664  
1      8482  
Name: count, dtype: int64
```

```
In [20]: sns.countplot(x="diabetes", data=df)  
plt.show()
```



```
In [21]: df.corr(numeric_only=True)
```

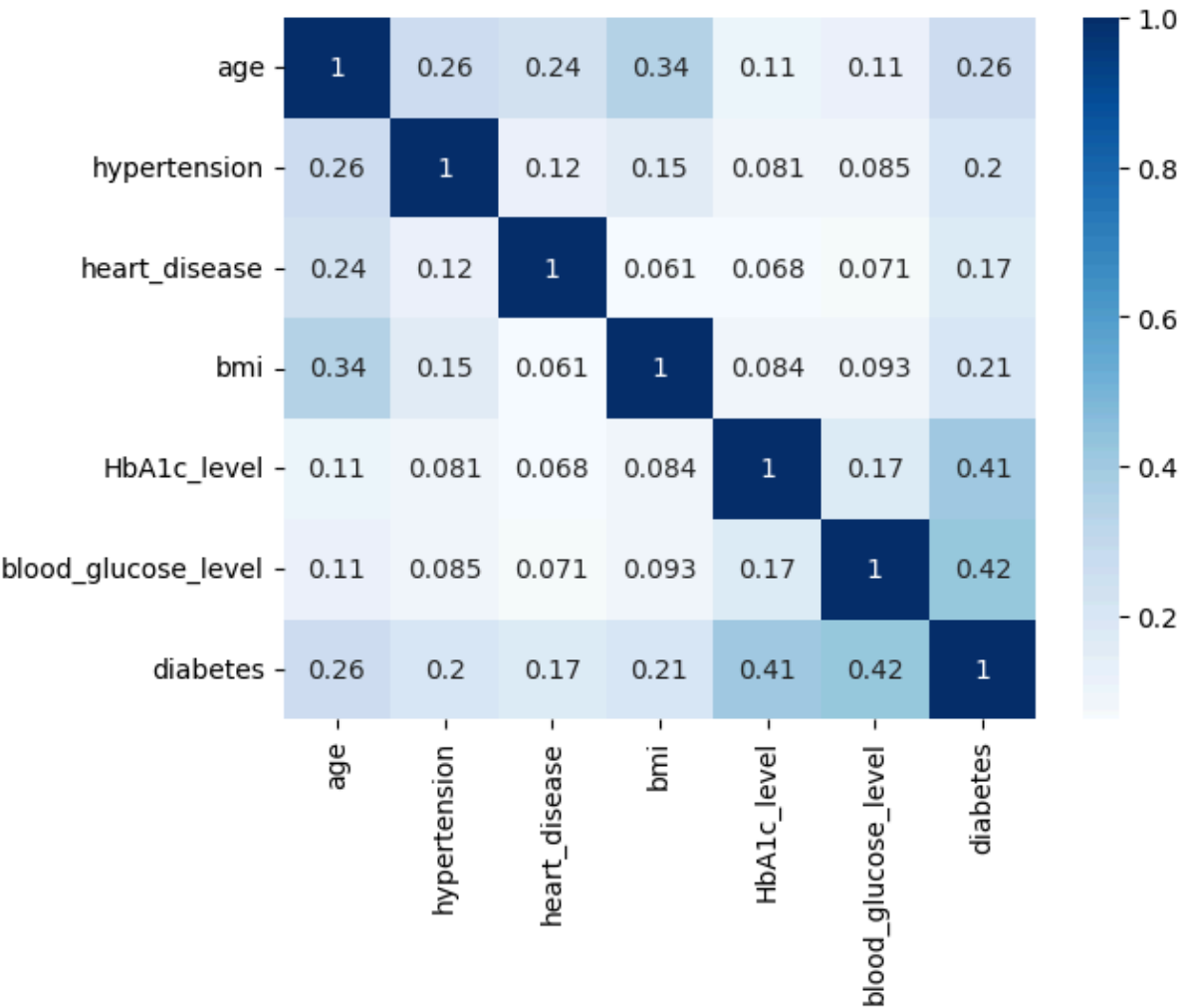
Out[21]:

	age	hypertension	heart_disease	bmi	HbA1c_level	blood_glucose_level
age	1.000000	0.257305	0.238451	0.344797	0.106719	0.114317
hypertension	0.257305	1.000000	0.119982	0.148111	0.081441	0.084834
heart_disease	0.238451	0.119982	1.000000	0.061376	0.068140	0.070832
bmi	0.344797	0.148111	0.061376	1.000000	0.084423	0.092543
HbA1c_level	0.106719	0.081441	0.068140	0.084423	1.000000	0.171615
blood_glucose_level	0.114317	0.084834	0.070832	0.092543	0.171615	1.000000
diabetes	0.264927	0.195710	0.170711	0.214932	0.406408	0.406408

Correction Heatmap

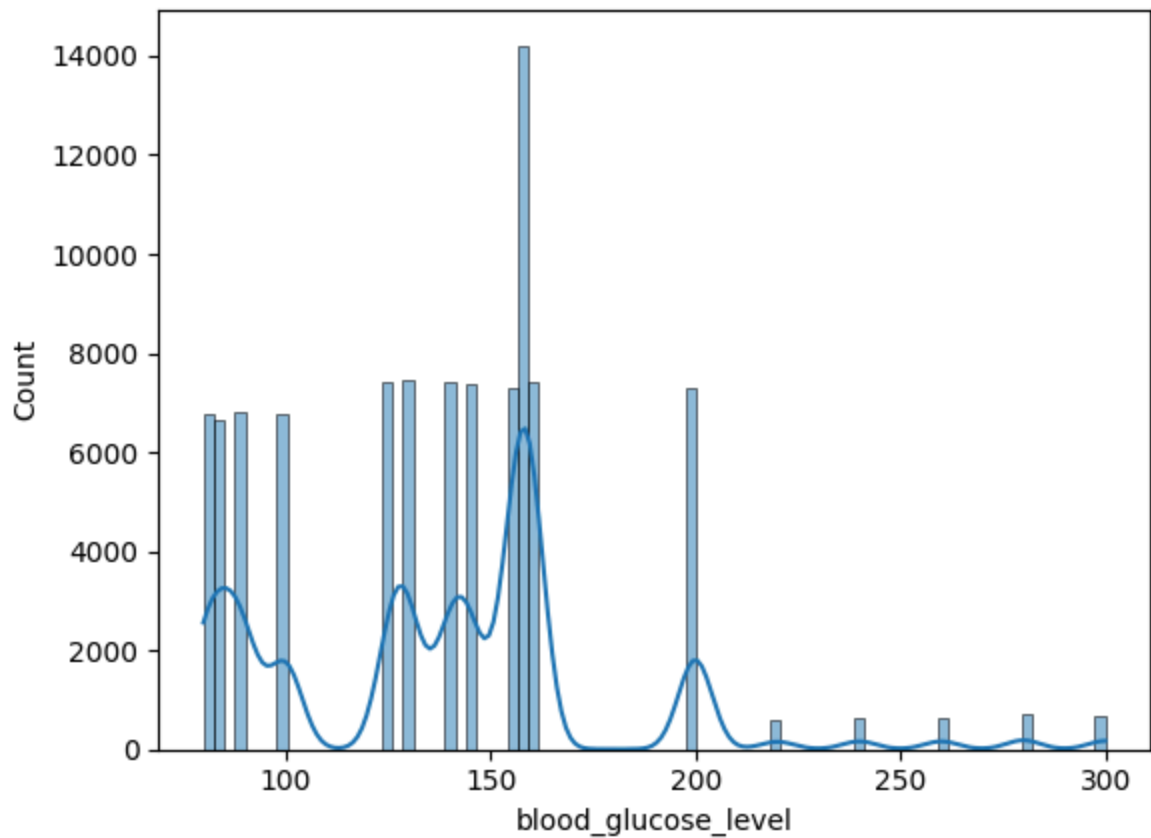
In [22]:

```
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="Blues")
plt.show()
```



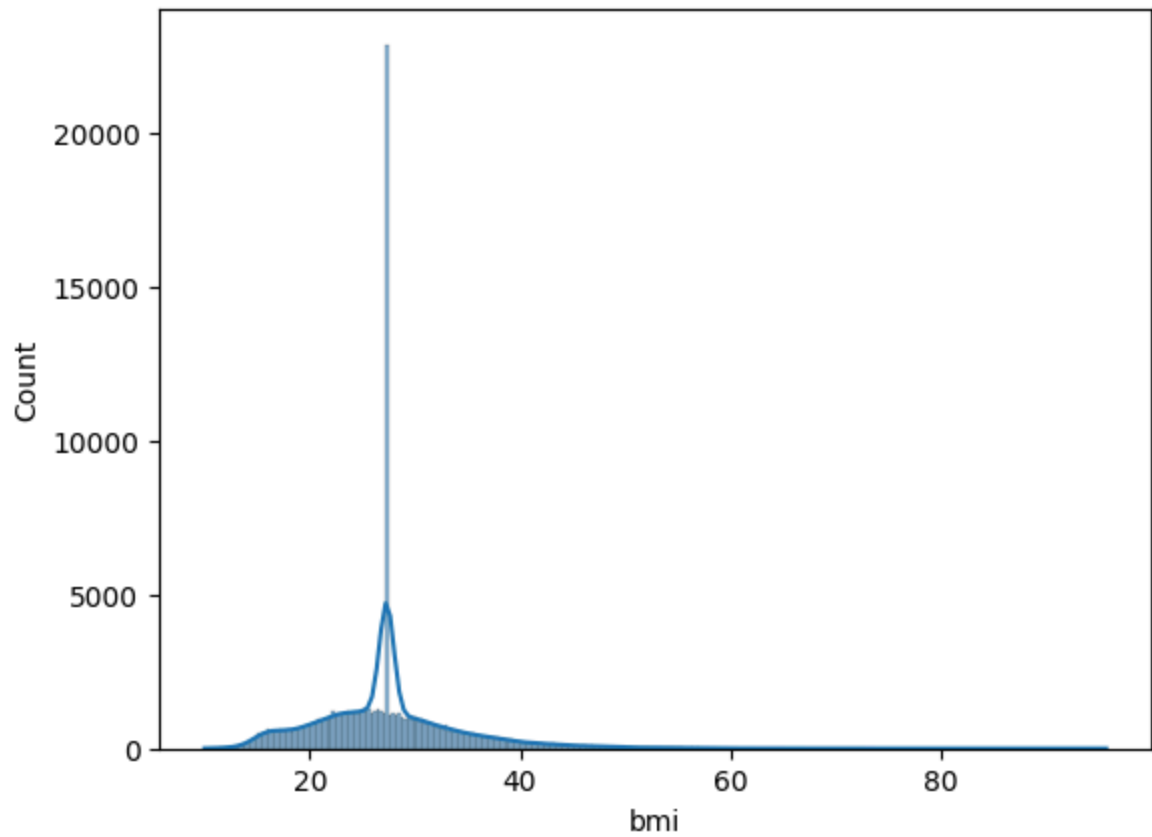
Histogram

```
In [25]: sns.histplot(df["blood_glucose_level"], kde=True)  
plt.show()
```



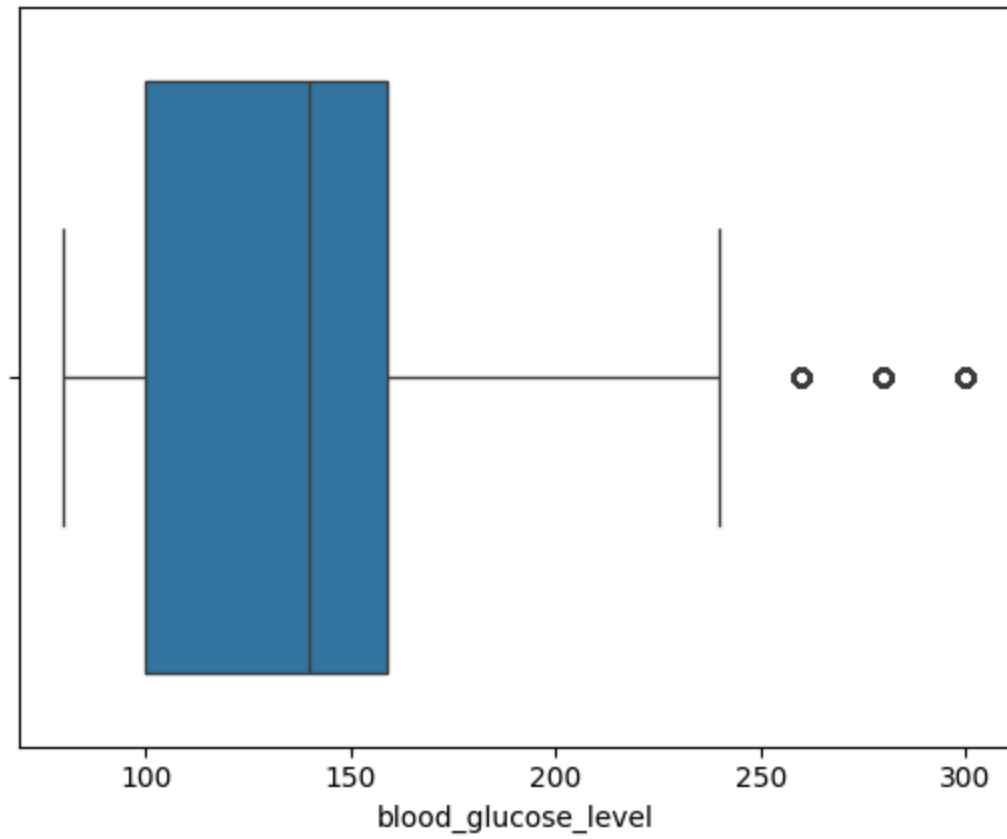
BMI

```
In [24]: sns.histplot(df["bmi"], kde=True)  
plt.show()
```

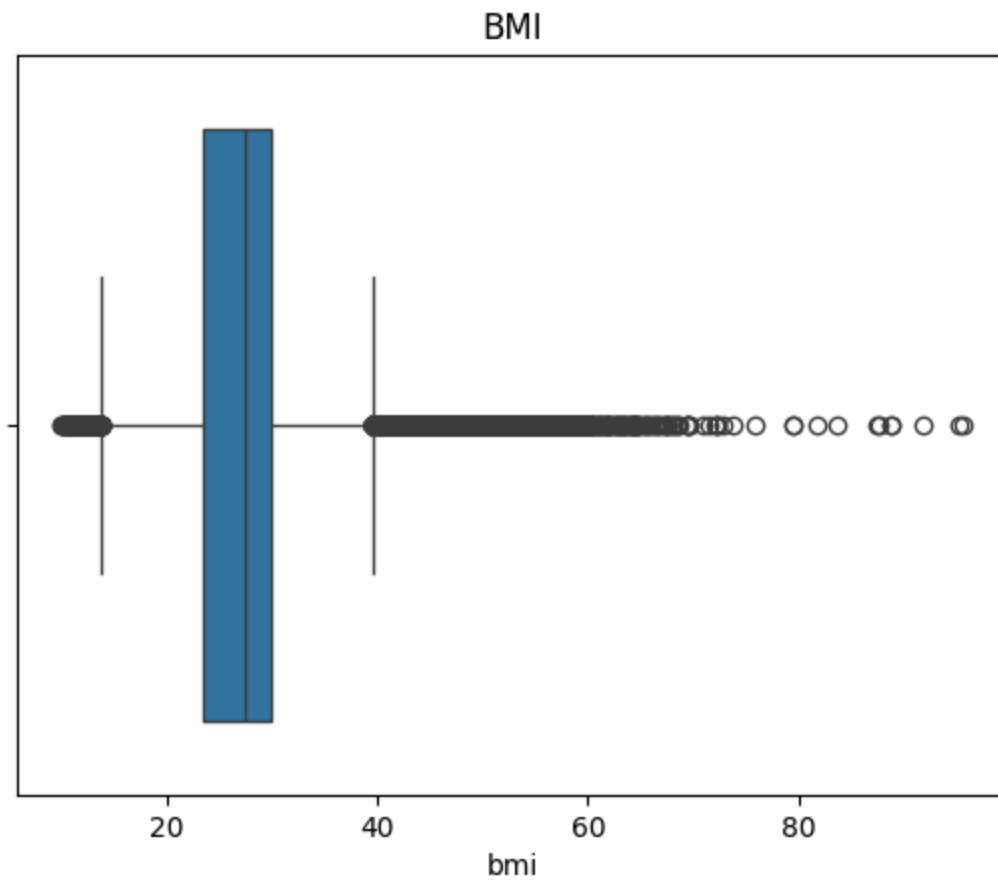


Outliers

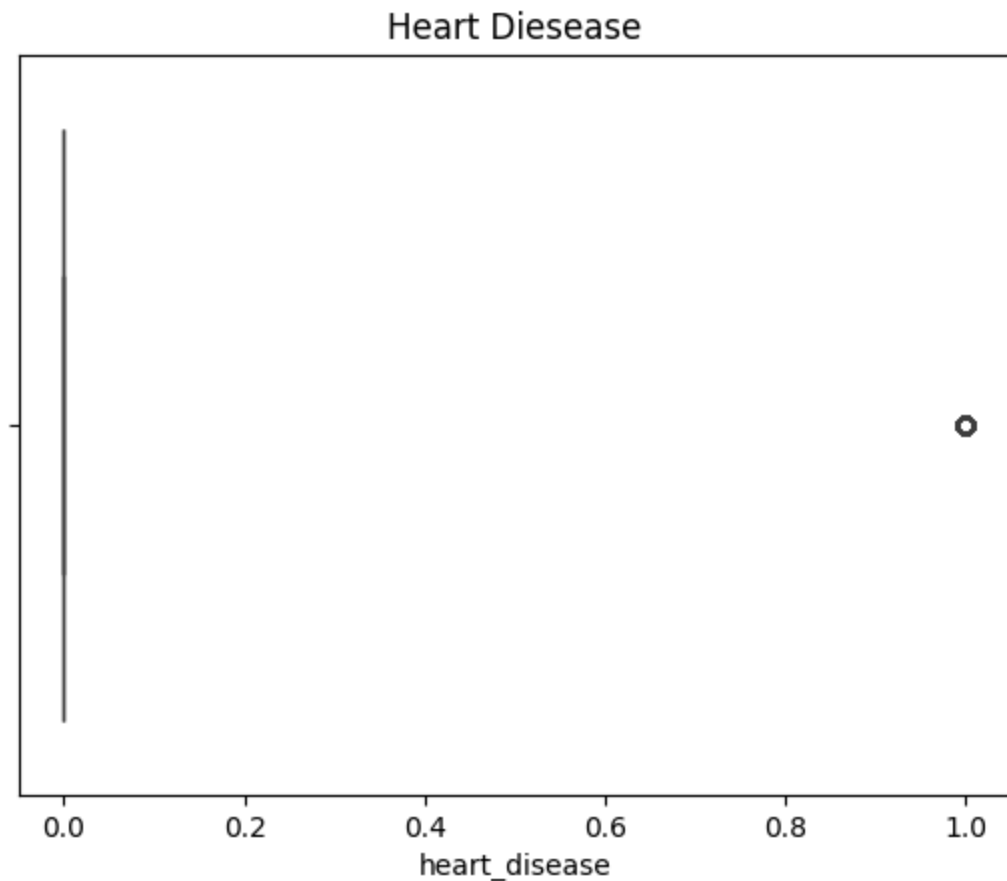
```
In [26]: sns.boxplot(x=df["blood_glucose_level"])  
plt.show()
```

```
In [32]: sns.boxplot(x=df["bmi"])  
plt.title("BMI")  
plt.show()
```



```
In [31]: sns.boxplot(x=df["heart_disease"])  
plt.title("Heart Disease")  
plt.show()
```



Feature Engineering

In [37]: `from sklearn.preprocessing import LabelEncoder`

`le = LabelEncoder()`

`df["gender"] = le.fit_transform(df["gender"])`

`df["smoking_history"] = le.fit_transform(df["smoking_history"])`

In [38]: `df.head()`

Out[38]:

	gender	age	hypertension	heart_disease	smoking_history	bmi	HbA1c_level	blood_g
0	0	80.0	0	1	4	25.19	6.6	
1	0	54.0	0	0	0	27.32	6.6	
2	1	28.0	0	0	4	27.32	5.7	
3	0	36.0	0	0	1	23.45	5.0	
4	1	76.0	1	1	1	20.14	4.8	



Model Building

```
In [39]: X = df.drop("diabetes", axis=1)
y = df["diabetes"]
```

Train-Test Split

```
In [40]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
In [44]: from sklearn.linear_model import LogisticRegression

model = LogisticRegression()

model.fit(X_train, y_train)
```

C:\Users\Admin\AppData\Local\Programs\Python\Python312\Lib\site-packages\sklearn\linear_model_logistic.py:599: ConvergenceWarning: lbfgs failed to converge after 100 iterations(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=100).

You might also want to scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

n_iter_i = _check_optimize_result(

```
Out[44]: LogisticRegression ⓘ ?
  ▸ Parameters
  ▸ Fitted attributes
```

Model Evaluation

Prediction

```
In [47]: y_pred = model.predict(X_test)
```

Accuracy

```
In [48]: from sklearn.metrics import accuracy_score

accuracy = accuracy_score(y_test, y_pred)
```

```
print("Accuracy:", accuracy)
```

Accuracy: 0.9509620384815393

```
In [50]: print("Accuracy:", accuracy * 100)
```

Accuracy: 95.09620384815392

```
In [51]: print("Accuracy:", round(accuracy * 100, 2), "%")
```

Accuracy: 95.1 %

```
In [49]: from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)

print(cm)
```

```
[[17266   243]
 [   700  1021]]
```

The Logistic Regression model achieved an accuracy of 95.10%, indicating that the model predicts diabetes with good performance

Conclusion

This project successfully predicts diabetes using Machine Learning. After preprocessing and training the Logistic Regression model, an accuracy of 95.10% was achieved. The model can help in the early prediction of diabetes..

```
In [ ]:
```